

Thomas Henderson ML Coursework 2

Statement on the use of GenAI: I used ChatGPT to summarise and aid me to analyse the documentation of numpy functions (numpy.random.shuffle, numpy.linalg.norm, numpy.array.split), matplotlib function (matplotlib.pyplot.semilogy seen in problem sheets), sklearn.svm.SVC and scipy function (minimize). This helped me understand the parameters and outputs of each function more clearly. I also asked it to explain the principles behind Stochastic Gradient Descent, what the parameters in the algorithm are and how they affect the algorithm itself which helped me to get an intuitive understanding of how the algorithm works and what might be the best parameters for different situations.

Question 4 Report

Q: Which value of t gives the fastest convergence? What makes smaller and larger t not optimal?

A: Running the algorithm 5 times for each t , the value of t that gives on average the fastest convergence was $t = 16$. $t = 1, 2$ and 3 all tended to never converge and so their average was around 840. $t = 64$ overshoot the optimal and diverged as the learning rate is too large. $t = 16$ and 32 had the best results with $t = 16$ slightly better on average.

Q: At which iteration k^ the algorithm stops with this optimal t ?*

A: For the optimal ($t = 16$), k_{star} is approximately 278 and the algorithm stops after approximately 348 iterations.

Q: What type of convergence can be seen in the loss plot?

A: The graph is roughly a reverse sigmoid function, or linear with plateaus at the beginning and end, with the linear trend starting on average at about $k = 40$ and ending at about $k = 220$, with the main trend between those values being very linear. As we are in the logarithmic scale this denotes an exponential decrease and so we can describe the convergence as exponential.

Q: What is the total relative error of the final result? How does it relate to the final test loss?

A: The average total relative error was approximately $1e^{-16}$ and the average test loss was approximately $2e^{-33}$, which is very close to the average total relative error squared. They were directly correlated when I ran sgd with the optimal t multiple times.

Question 5 Report

Method

I will use the same questions for Q4 for the report, but I will add detail of my approach. I started by running a test with every possible combination (excluding 64 as it raises

errors same as in Q4), then also excluded $t=1$ and $t=2$ as their loss values were too high. Note that 32 had a very small probability of diverging but as it happened incredibly few times, I decided it was still worth testing. I then plotted values of b against losses and time to converge for each t of the list $[4, 8, 16, 32]$. After that I found the best combination value for b for each corresponding t and then tested that with 5 iterations. Note that when choosing my final t and b I considered the minimal test loss, the average relative error and the number of iterations the algorithm took to converge.

Results

From my tests $t = 8$ and $b = 20$ gave the best results. This combination had an average loss of 0.045, average total relative error of 0.13 and stopped the algorithm after an average of 165 iterations which was the fastest out of the narrowed down list. The optimal $kstar$ was on average 147, also the lowest out of the narrowed down list. The average total relative error was about 3 times the average loss. In the plots for this combination there was an exponential with a negative power looking graph in the logarithmic scale. As the gradient is steeper than linear, this signifies that the loss is decreasing at a rate with a relationship that is faster than exponential. Even when the gradient plateaued the plateau was linear, suggesting that the loss was still decreasing at a low exponential rate.

Synthetic vs Real Data

The convergence in the plots was very different between the synthetic data and the real data, the decrease in the first few iterations was markedly greater for the real data, but there was greater variance per run of the sgd algorithm even for the optimised t and b . The plots were also much less smooth for the real data, suggesting that the synthetic Y function could be improved to model the real data better. Higher learning rates were best for the synthetic data as the optimal was $t=16$ and then second as $t=32$ compared to the real data which performed better with $t=8$ and $t=4$. Although the importance of batch size wasn't tested on the synthetic data, the best batch size for most t for the real data was 20 and not 10.

Question 7

Q7 Solution

Note: $\sum_{j=0}^{r-1} u_{i,j} w_{j,2} = \underline{u}_i \cdot \underline{w}_{i,2} = \underline{u}_i \underline{w}_{i,2}^T$

$$\frac{\partial \ell_{i,1,2,r}(u,w)}{\partial w_{i^*,j^*}} = \frac{\partial}{\partial w_{i^*,j^*}} \left(\left(\sum_{j=0}^{r-1} u_{i,j} w_{j,2} - y_{i,1,2} \right)^2 \right)$$

$$= 2(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}) \frac{\partial}{\partial w_{i^*,j^*}} \left(\sum_{j=0}^{r-1} u_{i,j} w_{j,2} - y_{i,1,2} \right)$$

$$= 2(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}) \sum_{j=0}^{r-1} \frac{\partial}{\partial w_{i^*,j^*}} (u_{i,j} w_{j,2} - y_{i,1,2})$$

$$= 2(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}) \sum_{j=0}^{r-1} \begin{cases} w_{j,2} & \text{if } i^* = i_1, j = j^* \\ 0 & \text{otherwise} \end{cases}$$

$$= 2(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}) \begin{cases} w_{j^*,2} & \text{if } i^* = i_1 \\ 0 & \text{otherwise} \end{cases}$$

$$\Rightarrow \frac{\partial \ell_{i,1,2,r}(u,w)}{\partial w_{i^*,j^*}} = \begin{cases} 2w_{j^*,2}(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}), & \text{if } i_1 = i^* \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Then } (\nabla_u \ell_{i,1,2,r}(u,w))_{ij} = \begin{cases} 2w_{j,2}(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}), & \text{if } i_1 = i \\ 0 & \text{otherwise} \end{cases}$$

i.e. $\nabla_u \ell_{i,1,2,r}(u,w)$ only has one non-zero row at row index i_1 . Row i_1 can be represented as the vector:

$$2 \underline{w}_{i,2} (\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}) = \underline{v}^u \text{ in cwnotes.}$$

Now wrt w :

$$\frac{\partial \ell_{i,1,2,r}(u,w)}{\partial w_{i^*,j^*}} = \frac{\partial}{\partial w_{i^*,j^*}} \left((\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2})^2 \right)$$

$$= 2(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}) \left(\sum_{j=0}^{r-1} \frac{\partial}{\partial w_{i^*,j^*}} (u_{i,j} w_{j,2}) \right) \left[\text{for } \frac{\partial}{\partial w_{i^*,j^*}} \right]$$

$$= 2(\underline{u}_i \underline{w}_{i,2}^T - y_{i,1,2}) \sum_{j=0}^{r-1} \begin{cases} u_{i,j^*} & \text{if } i^* = i_2, j = j^* \\ 0 & \text{otherwise} \end{cases}$$

$$\Rightarrow \frac{\partial \ell_{i,i_2,Y}(U,W)}{\partial w_{i,i_2}} = \begin{cases} 2u_{i,i_2}(\underline{u}_{i,i_2}^T - y_{i,i_2}), & \text{if } i_2 = i_2 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{Then } (\nabla_w \ell_{i,i_2,Y}(U,W))_{i,j} = \begin{cases} 2u_{i,i_2}(\underline{u}_{i,i_2}^T - y_{i,i_2}), & \text{if } i = i_2 \\ 0, & \text{otherwise} \end{cases}$$

i.e. $\nabla_w \ell_{i,i_2,Y}(U,W)$ has only one non-zero row at row index i_2 . Row i_2 represented by vector

$$2\underline{u}_i(\underline{u}_{i,i_2}^T - y_{i,i_2}) = \underline{v}^w \quad \square$$

Question 8

Q8 Solution

Show Algorithm 2 assembles gradient correctly.

$$\begin{aligned}\nabla_U L_{I,Y}(U,W) &= \nabla_U \left(\frac{1}{n} \sum_{(i_1, i_2) \in \mathcal{I}} \ell_{i_1, i_2, Y}(U, W) \right) \\ &= \frac{1}{n} \sum_{(i_1, i_2) \in \mathcal{I}} \nabla_U \ell_{i_1, i_2, Y}(U, W) \quad [\text{Linearity of gradient}]\end{aligned}$$

As $\nabla_U \ell_{i_1, i_2, Y}(U, W)$ is a matrix only non-zero for row i_1 which equals $\underline{v}^u$ from coursework notes which implies that $\nabla_U L_{I,Y}(U, W)$ can be represented as a zero matrix with $\underline{v}^u$ calculated and then added $\forall (i_1, i_2) \in \mathcal{I}$.

$$\begin{aligned}\nabla_W L_{I,Y}(U, W) &= \frac{1}{n} \sum_{(i_1, i_2) \in \mathcal{I}} \nabla_W \ell_{i_1, i_2, Y}(U, W) \quad [\text{Same as for } \nabla_U] \\ \nabla_W \ell_{i_1, i_2, Y}(U, W) &\text{ nonzero except for } \underline{v}^w \text{ at row } i_2 \text{ so apply same logic for } \nabla_U.\end{aligned}$$

Now let's compute $\nabla_U L_{I,Y}(U, W)$ and $\nabla_W L_{I,Y}(U, W)$ by some approach as Q7.

$$\begin{aligned}\frac{\partial L_{I,Y}(U, W)}{\partial u_{i_1, j_1}} &= \frac{\partial}{\partial u_{i_1, j_1}} \left(\frac{1}{n} \sum_{(i_1, i_2) \in \mathcal{I}} \ell_{i_1, i_2, Y}(U, W) \right) \\ &= \frac{1}{n} \sum_{(i_1, i_2) \in \mathcal{I}} \frac{\partial \ell_{i_1, i_2, Y}(U, W)}{\partial u_{i_1, j_1}} \\ &= \frac{1}{n} \sum_{(i_1, i_2) \in \mathcal{I}} \begin{cases} 2w_{i_2, j_1} (u_{i_1, i_2} w_{i_2, i_2} - y_{i_1, i_2}), & \text{if } i_1 = i_2 \\ 0, & \text{otherwise} \end{cases} \\ &= \frac{1}{n} \sum_{i_2 \in \{0, \dots, n-1\}} 2w_{i_2, j_1} (u_{i_1, i_2} w_{i_2, i_2} - y_{i_1, i_2}) \\ &= \frac{2}{n} \sum_{i_2 \in \{0, \dots, n-1\}} (u_{i_1, i_2} w_{i_2, i_2} - y_{i_1, i_2}) w_{i_2, j_1}\end{aligned}$$

$$\text{Consider } UW^T - Y: (UW^T - Y)_{ij} = \sum_{k=0}^{n-1} u_{i,k} (W^T)_{kj} - y_{i,j}$$

$$\begin{aligned}\Rightarrow \frac{\partial L_{I,Y}(U, W)}{\partial u_{i_1, j_1}} &= \frac{2}{n} \sum_{i_2 \in \{0, \dots, n-1\}} (UW^T - Y)_{i_1, i_2} w_{i_2, j_1} \\ &= \frac{2}{n} \left((UW^T - Y)_{\text{row } i_1} \cdot W_{\text{col } j_1} \right)\end{aligned}$$

$$\Rightarrow \nabla_U L_{I,Y}(U,W) = \frac{2}{n} (UW^T - Y)W \left[\begin{array}{l} \text{As } (\nabla_U L_{I,Y}(U,W))_{ij} \\ = \frac{\partial L_{I,Y}(U,W)}{\partial u_{ij}} \end{array} \right]$$

$$\begin{aligned} \frac{\partial L_{I,Y}(U,W)}{\partial w_{i_1, j_1}} &= \frac{1}{n} \sum_{(i_1, i_2) \in I} \frac{\partial}{\partial w_{i_1, j_1}} (\ell_{i_1, i_2, Y}(U, W)) \\ &= \frac{1}{n} \sum_{(i_1, i_2) \in I} \sum \left\{ \begin{array}{l} 2u_{i_1, j_1} (u_{i_1, i_2} - y_{i_1, i_2}) \text{, if } i_1 = i_2 \\ 0 \text{, otherwise} \end{array} \right\} \\ &= \frac{2}{n} \sum_{i_1=0}^{M-1} (u_{i_1, j_1} - y_{i_1, i_1}) u_{i_1, j_1} \\ &= \frac{2}{n} \sum_{i_1=0}^{M-1} (UW^T - Y)_{i_1, i_1} u_{i_1, j_1} \\ &= \frac{2}{n} ((UW^T - Y)_{\text{col } j_1} \cdot U_{\text{col } j_1}) \end{aligned}$$

$$\Rightarrow \nabla_W L_{I,Y}(U,W) = \frac{2}{n} ((UW^T - Y)^T U) \left[\begin{array}{l} \text{As } (\nabla_W L_{I,Y}(U,W))_{ij} \\ = \frac{\partial L_{I,Y}(U,W)}{\partial w_{i,j}} \end{array} \right]$$



Question 9

When researching strong convexity I found this set of notes with a remark useful to the proof, I referenced it in the proof itself.

<https://www.damtp.cam.ac.uk/user/hf323/L23-III-OPT/lecture3.pdf>

Q9 Solution

Assumptions: $\lambda_{\min}(W^T W) > 0 \Leftrightarrow W^T W$ positive definite

I contains all possible indices (i_1, i_2)
Let $\tilde{L}$ denote $\tilde{L}_{I,Y,W}(u)$ for convenience.

Remark 4:

One way of characterising λ -strong convexity when considering euclidean norm (the frobenius norm is the euclidean norm of a matrix) is that a function $\tilde{L}$ is λ -strongly convex iff $\nabla^2 \tilde{L} \geq \lambda I$ where I is the identity matrix. This is equivalent to the condition all eigenvalues of $\nabla^2 \tilde{L}$ are $\geq \lambda$.

As $\tilde{L}_{I,Y,W}(u)$ is a function of u , $\nabla^2 \tilde{L}_{I,Y,W}(u) = \nabla_u^2 \tilde{L}_{I,Y,W}(u)$

$$\begin{aligned}\nabla_u \tilde{L}_{I,Y,W}(u) &= \nabla_u L_{I,Y}(u, W) \\ &= \frac{2}{m} (u W^T - Y) W \quad [Q8]\end{aligned}$$

$$\begin{aligned}\nabla_u^2 \tilde{L}_{I,Y,W}(u) &= \nabla_u (\nabla_u \tilde{L}_{I,Y,W}(u)) \\ &= \nabla_u \left(\frac{2}{m} (u W^T - Y) W \right) \\ &= \frac{2}{m} \nabla_u (u W^T - Y) W \quad \left[\begin{array}{l} \text{matrix multiplication distributive} \\ \text{wrt matrix addition} \end{array} \right] \\ &= \frac{2}{m} \frac{\partial}{\partial u} (u W^T - Y) W \\ &= \frac{2}{m} (W^T W)\end{aligned}$$

All eigenvalues of $\nabla_u^2 \tilde{L}_{I,Y,W}(u)$ being $\geq \lambda \Leftrightarrow \lambda_{\min}(\nabla_u^2 \tilde{L}_{I,Y,W}(u)) \geq \lambda$

$$\lambda_{\min}(\nabla_u^2 \tilde{L}_{I,Y,W}(u)) = \lambda_{\min}\left(\frac{2}{m} W^T W\right)$$

For any eigenpair of $W^T W$: $(W^T W)v = \lambda_i v$,

$$(\nabla_u^2 \tilde{L}_{I,Y,W})v = \frac{2}{m} (W^T W)v = \frac{2}{m} \lambda_i v$$

$\Rightarrow$ For some eigenvector $\nabla_u^2 \tilde{L}_{I,Y,W}$ has eigenvalue $\frac{2}{m} \lambda_i$

$$\Rightarrow \lambda_{\min}(\nabla_u^2 \tilde{L}_{I,Y,W}(u)) = \frac{2}{m} \lambda_{\min}(W^T W)$$

So all eigenvalues of $\nabla^2 \mathcal{L}_{I,y,W}$ are $\geq \frac{2}{m} \lambda_{\min}(W^T W)$
 $\Rightarrow \mathcal{L}_{I,y,W}(u)$ is λ -strongly convex for $\lambda = \frac{2}{m} \lambda_{\min}(W^T W)$ $\square$

Question 10

Amendment 1: Adaptive learning rates

Using a constant for the learning rate t for algorithm 3 has two downsides. The first is that if the learning rate is too low, the algorithm will take a long time to converge, and the second is that if the rate is too high then it will overshoot the optimal and diverge. As our loss function is beta smooth for $2/m * (\text{the largest eigenvalue of } W^T W)$ and lambda strongly convex for $\lambda = 2/m * (\text{the smallest eigenvalue of } W^T W)$, we can choose $t_k = t_0 / k$ where k is the number of the iteration and t_0 was our initial t . This ensures convergence is quicker and prevents divergence because of the test loss increasing above the largest number representable on a computer, e.g. the errors I saw when I ran the algorithm for $t=64$.

Amendment 2: Stochastic Average Gradient (SAG)

Instead of calculating $\mathbf{v}^u$ and $\mathbf{v}^w$ for each k iteration, the SAG algorithm updates them each iteration using the pointwise loss gradient. As our loss function is beta-smooth for $2/m * (\text{the largest eigenvalue of } W^T W)$ and lambda strongly convex for $\lambda = 2/m * (\text{the smallest eigenvalue of } W^T W)$, it will converge q -linearly ($q = \sqrt{1 - \lambda / \beta}$). Crucially however, the time of one iteration will not grow with m as compared to Stochastic Gradient Descent and thus applying SAG instead of SGD would be better for larger datasets.

Amendment 3: Algorithm 2 storing whole matrix G_u and G_w

Instead of creating the zero matrices G_u and G_w and then having to fetch and update them (including every row) from memory every iteration, create two-dimensional lists g_u and g_w where the first index is the row number and the second is the vector at that index and update only the element corresponding to the correct row for each iteration of the for loop. This avoids the referencing of rows that are not updated when the I matrix does not contain all possible indices and so the more memory is saved the smaller that m is. This list can be referenced in all future calculations to save memory.